

Caracal — Como rodar o RL cyber (GRPO + RSI)

Notebook `notebooks/kaggle/ignite/caracal_rl_cyber.ipynb` · branch `s07-hybrid-agentic` · atualizado 2026-07-21

O que este run faz

Um único **Run All** no Kaggle mede a base, treina os dois métodos de RL sobre o Qwen2.5-Coder-3B, benchmarka cada um logo depois, e no fim roda a suíte cyber inteira comparando os três modelos lado a lado.

#	Etapas	Mede
1	Base no dev CVE→CWE	referência (~44%)
2	GRPO direto (RL puro)	curva de performance a cada 20 steps
3	RSI outer loop (same-model)	performance por geração (acc + hier + unparsed)
4	Benchmark final	suíte inteira: base vs melhor GRPO vs melhor RSI

Pré-requisitos

- Uma conta **Kaggle** com quota de GPU disponível (30h/semana por conta). Se a sua esgotou, use a de outro founder — o notebook clona tudo do GitHub, não depende de disco local.
- Nada mais: sem chave de API, sem dataset local. O checkpoint base baixa do Hugging Face automaticamente.

Passo a passo (Kaggle)

1. Abrir o notebook. `kaggle.com` → Create → New Notebook → File → Import Notebook → suba `caracal_rl_cyber.ipynb` (ou aponte pro repo GitHub `iterate-labs-ai/caracal-1`, path `notebooks/kaggle/ignite/`).

2. Configurar o ambiente (barra lateral direita).
Accelerator → **GPU T4 x2** · Internet → **ON** · Persistence → não precisa.

3. (Opcional) Ajustar o tamanho do run — primeira célula de código:

```
GRPO_STEPS = 80           # steps de RL direto
RSI_GENS, RSI_CANDS, RSI_STEPS = 2, 2, 10 # outer loop
```

Padrão cabe nas 12h de sessão. Aumente só se tiver folga de quota.

4. Rodar. **Save Version** → **Save & Run All (Commit)** — roda em batch no servidor (não precisa deixar a aba aberta). Ou Run All interativo pra acompanhar ao vivo.

5. Ler os resultados. Quando terminar, abra a versão salva → Output. As tabelas aparecem no log das células 2, 3 e 4; os números crus ficam em `final_benchmark.json`, `rl_cyber_grpo/curve.json` e `rl_cyber_rsi/trajectory.json`.

Como ler os números

Marco	Valor	Significado
base	~0.44	o modelo sem treino no CVE→CWE. Ponto de partida.
piso de colapso	0.273	responder sempre CWE-79 (classe majoritária). Resultado perto disso com <code>unparsed</code> baixo = colapso, não aprendido.
alvo	0.72–0.75	Foundation-Sec-8B RCM. O teto do specialist de 8B. Headroom ~28pp.

As três métricas por ponto

- **acc** — acerto exato do CWE. O número comparável.
- **hier** — crédito parcial na árvore CWE (acerto do ancestral vale 0.5, irmão 0.3). Pega movimento fino que o acerto exato ainda não capturou.
- **unparsed** — fração de respostas sem CWE nenhum. **Canária**: se sobe, a truncagem de 160 tokens apertou demais — é problema de formato, não o modelo piorando.

Interpretando o resultado

A pergunta que o run responde: o RL melhorou o CVE→CWE **sem** regredir o resto? O adapter anterior (s04) subiu 0 e **piorou** o RCM em -4.6pp. Se GRPO ou RSI passam da base no CVE→CWE **e** não afundam os outros benches da suíte final, aí sim é ganho real.

- **Curva/trajetória sobe da base e unparsed fica baixo** → aprendizado real.
- **acc cai pra perto de 0.273** → colapso na classe majoritária. Reduzir lr ou steps.
- **acc sobe mas unparsed dispara** → truncagem apertou; subir `max_completion_len` no `BENCH_GEN_BUDGET`.
- **GRPO sobe e RSI não** → o outer loop não está agregando; rodar mais gerações ou revisar o gate de retenção.

Se a quota esgotar no meio

O GRPO é resumível: re-rodar o notebook na mesma conta (com o output persistido) continua da última curva salva. O RSI retoma da última geração via `state.json`. Em conta nova, começa do zero (o clone do GitHub é sempre limpo).

Rodar local ou em outra plataforma

Os drivers são independentes do Kaggle:

```
python -m train.ignite.B_grpo_straight --total-steps 120 --out runs/grpo # GRPO direto
python -m train.ignite.C_rsi_outer --base Qwen/Qwen2.5-Coder-3B-Instruct \
  --bench cyber_rcm --bench-name cyber_rcm \
  --dataset-train data/ignite/cyber_rcm_train.jsonl \
  --dataset-dev data/ignite/cyber_rcm_dev.jsonl \
  --dataset-val data/ignite/cyber_rcm_val.jsonl \
  --gens 2 --cands 2 --steps 10 --out runs/rsi # RSI
```

Precisa de GPU NVIDIA com o stack pinado (torch 2.6.0 cu124, transformers 4.49, peft 0.14, trl 0.15.2). O dataset se constrói com `python -m data.ignite.build_cyber_rcm`.

Gerado para Iterate Labs · Caracal-1 · s07-hybrid-agentic. Números de referência medidos em 2026-07-20 (T4 x2). Documentação técnica completa: [docs/research/CARACAL_PAPER.md](#) e [docs/s07/CYBER_BASELINES_VERIFIED.md](#).